MedGov-AI: An MCP-Based Agentic Framework for Clinical AI Integration and Medical Assistance

Author Name

Institution, City, Country

Abstract

Clinical AI tools for medical imaging are increasingly capable but remain largely isolated from one another, requiring custom integration work that slows adoption and creates fragmented, difficult-to-maintain deployments. This paper presents MedGov-AI, an agentic AI system that addresses the integration and orchestration problem directly by using the Model Context Protocol (MCP) as a standardized interoperability layer. Rather than hardcoding workflow logic, MedGov-AI uses a large language model as an autonomous orchestrator that discovers and invokes clinical tools at runtime through a set of MCP servers covering DICOM parsing, CT segmentation (MONAI), whole-slide image cell segmentation (Cellpose), radiology report generation via the RadLex ontology, and digital pathology (iPath). Domain-specific workflow governance is provided through skill files, plain-text protocol documents that encode step-by-step clinical sequences and guard conditions without retraining the model. The system supports both interactive and event-driven operation: in workspace monitoring mode, files deposited into a configured directory trigger autonomous agent execution without any user input. Three end-to-end use cases were evaluated: DICOM-based CT analysis with structured report generation; interactive whole-slide pathology tumor detection via iPath with cell segmentation and structured pathology reporting; and autonomous workspace monitoring with Cellpose cell segmentation and cross-workspace file routing. Results show that API-hosted large language models complete multi-step clinical workflows reliably within a bounded iteration budget, while locally-hosted 7-8B parameter models do not, revealing a current tension between privacy and reliability in clinical deployment. Skill-guided execution prevented domain-incorrect tool selection in the pathology use case, confirming that skills are necessary at decision forks where plau-

sible but clinically incorrect options exist. Assisted and autonomous modes produced identical tool sequences. Six MCP servers were integrated without any agent-side connector code, supporting the argument that MCP can reduce the integration overhead that currently limits clinical AI adoption. The system is publicly available at `https://medgovai.bmd-software.com/`.

Keywords: Agentic AI, Medical Imaging, MCP, MONAI, Clinical Workflow Automation

1. Introduction

1.1. Positioning of This Work and Contribution

Prior work treats these components in isolation: MCP is evaluated as a protocol but not in clinical imaging workflows [1, 2]; conceptual frameworks identify fragmentation but stop short of implementation [3]; medical-imaging AI work focuses on accuracy rather than orchestration [4]; and skills have been formalized but not applied to clinical governance or evaluated against a no-skill baseline [5]. MedGov-AI combines DICOM parsing, segmentation inference, and structured reporting under a single agentic orchestrator using MCP as the interoperability layer and skill files for workflow governance, evaluating both in end-to-end workflows and providing empirical evidence for properties prior work argued only conceptually.

2. Related Work

2.1. AI in Clinical Practice

Healthcare systems face rising imaging volumes against a constrained supply of specialists, creating diagnostic backlogs that affect patient outcomes. Automating routine analysis is therefore a structural necessity: it improves consistency, reduces clinician workload, and frees experts for complex cases. A wide range of AI tools is already in clinical use, but with the notable characteristic that they typically operate in isolation, with no unified integration layer. In radiology, AI supports worklist prioritization and critical-finding alerts; in screening it acts as a first filter that lets programs scale [6, 7]. Mayo Clinic has integrated AI into ECG and cardiac imaging as decision support [8], and in pathology whole-slide analysis is used for tumour grading and cell counting. For segmentation and detection specifically, FDA-cleared

tools such as Viz.ai [9], Aidoc [10], Transpara [11], and IDx-DR [12] are deployed across triage, screening, and grading.

These benefits carry significant risks. Bias arises when models trained on specific populations underperform elsewhere [13], and can be introduced at any pipeline stage from dataset composition to deployment [14]. Hallucination is critical in report-generation models, where confident but incorrect statements carry direct patient risk [15], and the black-box nature of deep learning raises accountability concerns [16]. No unified framework governs how clinical AI is validated, updated, or audited in production, and approval at development time does not cover hazards emerging after deployment [17]. Most relevant here, no established standards govern how agentic systems operate in clinical environments or log decisions for audit [18]. The capability-to-deployment gap is substantial: fewer than 4% of AI studies have been evaluated in real clinical settings [19], with limited daily use reported among radiologists [16, 20].

Despite progress in individual tools, no standard layer orchestrates them within a workflow, forcing each institution to build and maintain its own connectors, which drives fragmentation, slow adoption, and costly maintenance. The Model Context Protocol (MCP) is a promising solution, providing a standardized way for agents to discover and call tools without custom integration, and has begun attracting clinical application [3, 21, 22]. This work extends that direction to multi-tool clinical imaging orchestration.

2.2. Agentic AI in Healthcare

Agentic AI refers to systems that autonomously set goals, plan multi-step actions, and use tools to achieve them [23]. Terminology is not standardized, some reserve the term for multi-agent orchestration, others apply it to any autonomous multi-step tool use [24]; we adopt the broader definition. The application of agentic AI in healthcare is still early, with reviews identifying very few systems evaluated in real settings [23, 25, 26]. The case for it rests on properties intrinsic to computation: systems do not fatigue, apply criteria uniformly, and process information faster than humans. Clinical imaging workflows follow defined protocols with repetitive, predictable steps and well-defined outputs, making them strong candidates for autonomous orchestration.

We evaluated existing frameworks including LangChain [27], CrewAI [28], and AutoGen [29], but built a custom agentic system because none natively support MCP as a tool protocol; adapter layers would have reintroduced

the fragmentation we sought to remove. A custom loop also let us enforce clinical behaviours such as a completion contract (the agent cannot declare success while deliverables remain unmet) and an optional per-call confirmation mode. We use a single-agent architecture, which offers greater simplicity, predictability, and accountability than multi-agent designs [23] - properties that matter where every tool call must be traceable. Our system began as a hardcoded pipeline in which the LLM was never consulted and all sequencing was explicit conditional logic; this could not adapt without reprogramming, motivating the transition to a fully agentic design. The fundamental advantage is scalability: supporting a new domain requires only an MCP server exposing the relevant tools - community servers already exist for standards such as FHIR [22] - and the agent incorporates new servers at runtime without changes to the orchestration layer.

We explored two backend approaches. The local option used Ollama's deepseek-r1:7b [30] and llama3.1:8b [31], small enough for consumer hardware and offering full data privacy. The local approach struggled with multi-step workflows: models frequently failed to select the correct tool and could not reliably chain calls, a limitation rooted in model scale rather than configuration. API-hosted models (Gemini 2.0/2.5 Flash, 3.0 [32]) are also a more accessible path for institutions that cannot justify GPU infrastructure [4], and resolved these issues immediately, at the cost of latency, third-party data transmission, and a dependence on external availability. Seamless, secure interoperability with existing platforms is a recurring prerequisite for agentic AI in care delivery [25], a problem MCP directly addresses.

2.3. MCP as a Standardized Tool Integration Layer

The conventional approach to agent-tool integration is the custom REST adapter, written per tool to handle authentication, serialization, parsing, and error mapping. This does not scale: each new tool adds an adapter, API changes break adapters, and the agent must encode assumptions about every tool. MCP defines a client-server protocol in which each server declares its tools at startup; the agent discovers and invokes them by name at runtime, decoupling it from individual tool APIs so that adding a server requires no agent changes. Singh et al. survey MCP's architecture as the solution to fragmented integrations, and identify MCP sampling (LLM-to-LLM communication within the protocol) as a path toward sub-agent specialization [1], noting healthcare's relevance but presenting no implemented system. A preliminary integration demonstrated 95% correct tool selection from natural-language

queries [2], and MCP has been proposed conceptually as a foundational layer for clinical AI [3], but production-grade clinical deployments remain scarce.

2.4. Skills as Clinical Capability Packages

Tool access via MCP determines what an agent *can* do, not what it *should* do in a given domain: the correct sequencing of segmentation, terminology, and templating tools for a CT lung evaluation versus a pathology workflow is domain knowledge separate from both the agent's reasoning and the tools' logic. Skills address this layer. Xu and Yan formalize agent skills as composable, on-demand capability packages that guide an LLM without retraining [5], noting that skills and MCP are complementary: skills define what to do, MCP exposes the tools to do it. They do not address healthcare-specific skill design, governance when protocols change, or composition with MCP in a working system - gaps this work addresses.

3. Methods

3.1. System Architecture Overview

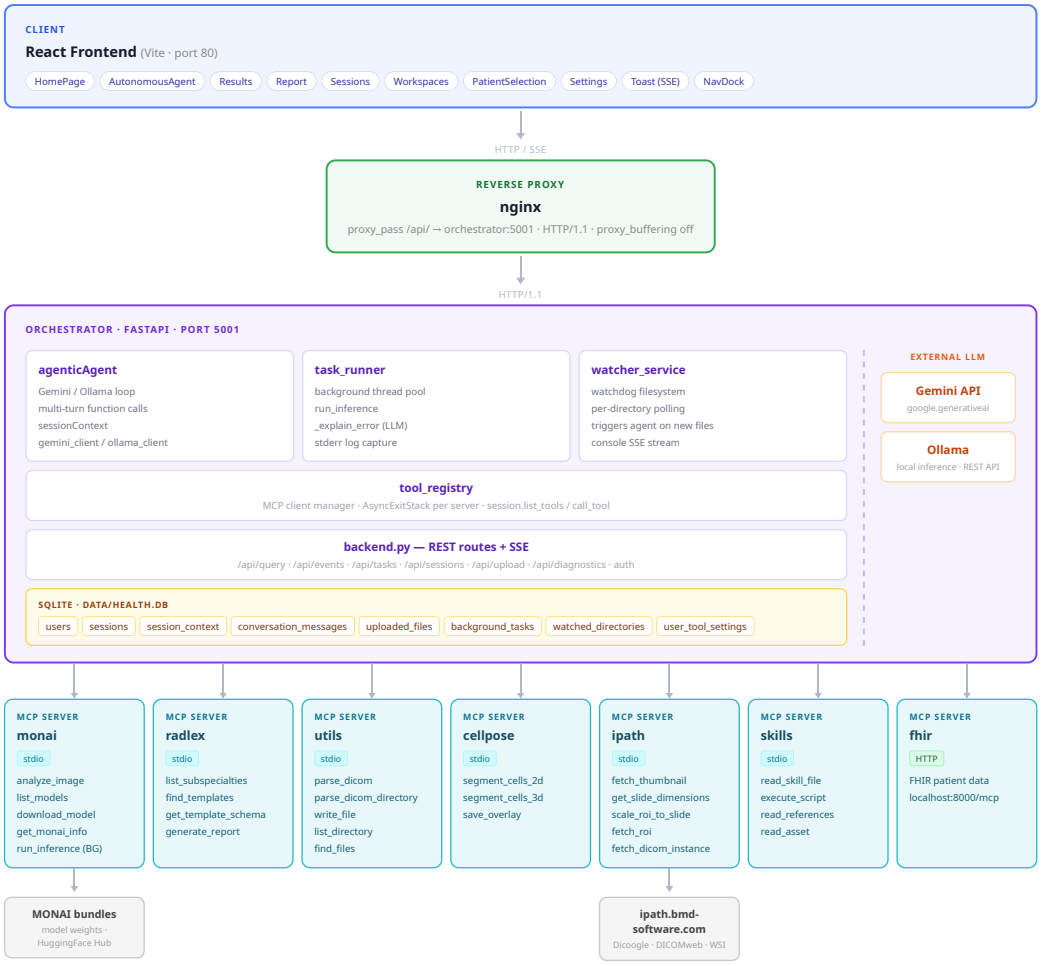


Figure 1: MedGov-AI system architecture.

MedGov-AI is organized into three layers - a React frontend, a FastAPI orchestrator, and a set of MCP tool servers (Figure 1) - with boundaries designed so that clinical domain logic lives entirely in the orchestrator, where it can be reasoned about, logged, and audited. The **frontend** is a React 18 application communicating with the orchestrator via a REST and SSE API; SSE streams task progress and agent responses without polling.

The **orchestrator** is a FastAPI application and the system's single point of control: it owns all session and task state, manages connections to every MCP server, and hosts three services. The `Agent` is the sole decision-maker; we use a single agent rather than a hierarchy because multi-agent coordination introduces inter-agent communication and shared-state synchronisation [33, 34], while our regime gains nothing from partitioning. A single agent already achieves specialisation through MCP and keeps every tool call in one auditable session trace [23]. It runs an iterative LLM loop (Gemini API or local Ollama) executing function calls across turns, maintaining session context so each result is available to the next. The `task_runner` manages long-running operations via a background thread pool tracked through a queued/running/completed/failed state machine; a semaphore limits concurrent GPU inference to one job, preventing out-of-memory crashes, and on failure passes captured stderr to the LLM for a plain-language explanation. The `watcher_service` polls registered directories and triggers the agent automatically when a file is deposited, enabling event-driven workflows from a PACS or scanner with no upstream changes. All state is persisted in a single SQLite database with write-ahead logging.

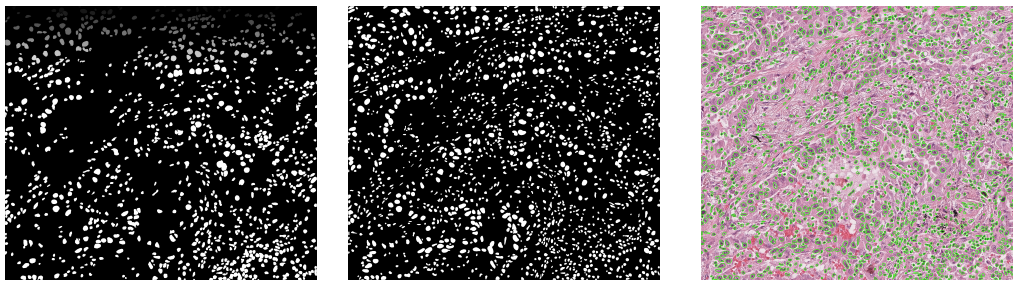
The orchestrator supports two **LLM backends**, selectable at deployment without changing agent logic: the Gemini API for reliable multi-step function calling at the cost of external data transmission, and Ollama for local inference with no external transmission - relevant in privacy-sensitive settings, but insufficient for reliable tool calling at the 7–8B scale available on ordinary hardware. The **tool layer** consists of six MCP servers, each an independent process in its own virtual environment; process isolation prevents dependency conflicts (most critically the GPU-enabled PyTorch required by `mcp-monai` versus the CPU-only install used elsewhere). Tool names are auto-prefixed by server (e.g. `monai.run_inference`), providing unambiguous namespacing. The six core servers are: `mcp-monai` (medical image segmentation and detection), `mcp-radlex` (RadLex terminology and report templates), `mcp-utils` (DICOM parsing and filesystem operations), `mcp-skills` (skill workflow loading and execution), `mcp-ipath` (iPath telepathology integration), and `mcp-cellpose` (2D/3D cell segmentation via Cellpose). An optional `fhir-mcp` server provides patient record retrieval via FHIR over HTTP.

FHIR-based patient record integration was explored by connecting an existing community FHIR MCP server rather than building one [22], demonstrating that third-party servers plug into the architecture without changes to

the orchestration layer. Extending the system thus requires only registering a compatible MCP server in the configuration file; the agent, orchestrator, and existing servers remain unchanged. This is the property that makes MCP a viable interoperability layer rather than a bespoke integration approach.

3.2. MCP Server Layer

Each server encapsulates a distinct capability as named, schema-defined tools that the agent discovers at startup and selects among by context. **Mcp-monai** is the inference server, built on MONAI [35] with PyTorch; it detects modality and body part from DICOM metadata (with pixel statistics as a fallback heuristic), returns a ranked list of compatible models retrieved from HuggingFace Hub on demand, and runs inference with a sliding-window strategy that applies the model to overlapping sub-volumes to avoid GPU memory overflow on large 3D scans. Bundled models cover spleen, pancreas/tumour, lung-nodule, and whole-body (104 structures) CT segmentation, brain-tumour MRI, and UNEsT kidney segmentation. **Mcp-radlex** provides structured radiology reporting via the RadLex terminology standard; when populating a report it uses MCP sampling to delegate the mapping of segmentation outputs to template fields back to the host LLM, which also generates the findings narrative, closing the loop between inference and standardized documentation. **Mcp-utils** provides DICOM metadata extraction (per file or directory) and general filesystem operations (read, write, move, copy, delete, list, search). **Mcp-cellpose** provides cell segmentation using the Cellpose v4 universal model (cpsam); v4 replaces v3's task-specific models (cyto3/cyto2/nuclei, ≈80 MB each) with a single SAM-based model (≈2.5 GB). On the same ROI, v3 (cyto) detected 1,123 cells and v4 (cpsam) 1,706; visual inspection confirmed v4 produced more complete boundaries in densely packed regions (Figure 2), so v4 was selected. The current implementation uses grayscale mode (fluorescence-based segmentation is a planned extension) and detects GPU at startup with CPU fallback, practical only for small images (Section 4). **Mcp-ipath** integrates the iPath telepathology platform, aggregating whole-slide images (WSI) and DICOM via Dicoogle and DICOMweb, supporting thumbnail retrieval, resolution-level coordinate mapping, and region-of-interest extraction without downloading the full WSI. **Mcp-skills** exposes the skill system as callable tools for loading a skill's entry point, references, and predefined scripts (Section 3.3).



v3 cyto (1,123 cells) v4 cpsam (1,706 cells) v4 cpsam overlay on H&E

Figure 2: Cellpose version comparison on the same pathology ROI. Binary segmentation masks from v3 (cyto) and v4 (cpsam) are shown alongside the v4 mask overlaid on the original H&E (Hematoxylin and Eosin) image. v4 produces denser and more complete cell detection in packed regions.

3.3. Skills System

The skills system encodes domain-specific guidance the agent loads on demand. A skill body can be a step-by-step workflow protocol with sequencing constraints and cross-server coordination, or a body of domain knowledge such as regulatory standards or report templates. This guidance that cannot be derived from tool schemas alone and is too domain-specific or voluminous to embed permanently in the system prompt. Embedding it permanently would increase token consumption on every request and raise the risk of instructions being overlooked in a large prompt.

A skill separates *awareness* from *detail*: every skill is surfaced at startup through a short name and description kept in the system prompt, while the full body stays unloaded until needed. When a request arrives, the agent reasons over these descriptions, identifies the applicable skill, and loads its body as operating instructions, enabling progressive disclosure, a lightweight index at rest, expanding to full detail only for the domain in use. A skill body can itself be hierarchical, routing from an entry document to sub-documents, references, templates, and scripts loaded independently as needed. Four skills are deployed: **dicom-analysis** encodes the tool sequence and guard conditions for DICOM analysis ending in a radiology report; **wsi-tumor-detection** encodes the inspection and cell-counting workflow for WSI pathology; and the community skills **clinical-reports** (regulatory standards and structured documentation templates) and **pydicom** (API guidance for DICOM manipulation) illustrate skills as knowledge packages adoptable without modification. A key governance property is that skill files are plain text and can

be updated without touching the agent, servers, or codebase: editing the relevant reference document changes behaviour on the next session with no software deployment, separating workflow maintenance from software development - relevant where the people who understand a procedure are not those who maintain the code.

3.4. Agentic Execution Loop

The agent is governed by an iterative loop that drives the system from a natural-language goal to completed deliverables, calling MCP tools based on the LLM's reasoning at each step. At session start the agent receives a system prompt establishing its role, tools, and rules, including the available skills (name and description) injected after dynamic discovery, and explicit guidance on when to read a skill before acting, when to queue long-running operations, how to handle DICOM directories versus files, and how to avoid repeating failed calls. Two prompt variants exist, for autonomous and assisted modes.

In each iteration, the agent receives the goal, available files, and the results of prior tool calls - the first iteration carries only the goal and uploaded files. The LLM responds with one or more function calls, executed sequentially within the iteration, or a text response signalling completion or a need for more information. A *repetition guard* addresses an observed failure mode in which the agent repeated the same failing call rather than adapting: if a tool fails twice in a row, the third attempt is blocked and an error is injected instructing the agent to try a different approach. Alongside MCP tools, built-in tools handled by the orchestrator give the agent control over its lifecycle: `goal_achieved` (the only way to exit successfully, carrying a summary), `need_more_info`, `queue_task`, and `list_tasks`. The system supports two operating modes: in **autonomous mode** the agent executes calls without interruption (the intended mode for event-driven workflows), while in **assisted mode** the loop pauses before each call and returns a `confirmation_required` response for user approval.

3.5. Background Task Queue and Workspace Monitoring

Inference operations are long-running, so blocking the agent or HTTP request would make the system unresponsive. All inference calls are offloaded to a background thread pool: the tool returns immediately with a task identifier and computation runs asynchronously, following a queued/running/completed/failed state machine persisted in SQLite so results survive restarts. A semaphore

limits concurrent inference to one job, and on completion an SSE notification updates the frontend without polling.

Not all clinical workflows are user-initiated: in many settings files are deposited into shared directories by existing institutional processes with no operator present. The workspace monitoring system turns filesystem events into agent triggers without modifying upstream workflows. Each monitored directory is a *workspace*: a named, registered directory stored in SQLite with a configurable prompt defining the agent's behaviour on arrival. The `watcher_service` polls each path recursively, bridging events into the orchestrator's event loop. On detecting a new file it starts a three-second debounce window - batching additional arrivals - so that a DICOM series arriving slice by slice does not spawn redundant executions. When the window closes, the watcher stages the files in the workspace's `incoming/` subfolder and constructs a goal prompt (workspace name, received files, preconfigured prompt), then invokes the agent in autonomous mode. The full activity log streams to the frontend via SSE under the workspace's console channel, giving real-time visibility even without user interaction.

3.6. Evaluation Design

Three clinical workflows were selected to cover the two target modalities (radiology and pathology) and exercise the full tool stack: DICOM parsing, segmentation inference, report generation, WSI analysis, and event-driven monitoring. Use Case 1 targets radiology; Use Cases 2 and 3 target pathology on WSI, differing in trigger - interactive and autonomous, respectively.

3.7. Evaluation Setup

Evaluation was structured in three phases. Phase 1 evaluates API-based and local models for tool-calling reliability. Phase 2 evaluates autonomous operation: how skills provide the constraints that make autonomous execution reliable, and whether assisted and autonomous execution of the same task produce equivalent results. Phase 3 runs all three use cases end to end, evaluating Use Cases 1 and 2 by task completion and qualitative output assessment, and Use Case 3 by end-to-end autonomous operation of the monitoring pipeline. Language-model experiments ran on a laptop (AMD Ryzen 5 8645HS, 16 GB DDR5, NVIDIA RTX 3050 6 GB). MONAI benchmarks ran on the same machine using CUDA and on a shared CPU-only deployment server (MONAI 1.5.1, PyTorch 2.6.0). Two model categories were tested:

large API-based proprietary models (Gemini 2.0 Flash, 2.5 Flash, 3.0; parameter counts undisclosed) and locally hosted open-source models (≈7–8B: deepseek-r1:7b, llama3.1:8b, via Ollama).

3.8. Use Case 1 - Interactive DICOM Analysis and Segmentation

A clinician has a DICOM scan and needs segmentation without knowing which model to use or how to chain tools. Success requires a correctly identified study type, a completed segmentation, and a filled report template with findings mapped to the right fields. The user uploads a DICOM volume or series and prompts: *"Analyze this DICOM series and run the appropriate segmentation model. Make a report with the findings."* The workflow (Figure 3): the agent calls `parse_dicom_file` for header metadata and `analyze_image` for image type and dimensions, then `list_models` for compatible models, `download_model` if not cached, and `run_inference` (queued to the background, sliding window). It responds immediately after queuing; on completion, SSE pushes per-organ volumes to the Results tab.

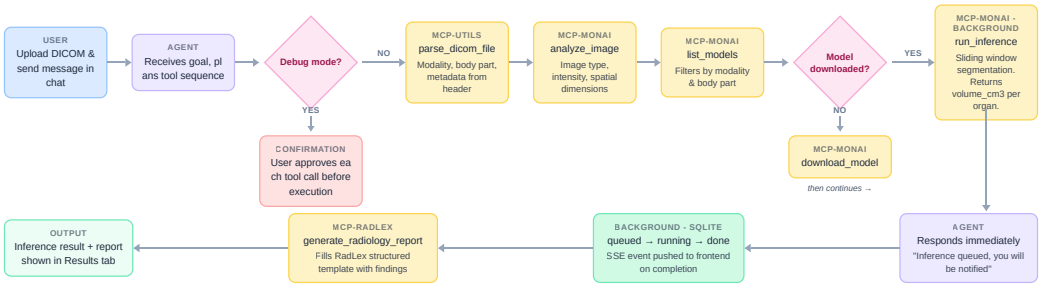


Figure 3: Use Case 1 - Interactive DICOM analysis and segmentation workflow.

3.9. Use Case 2 - Interactive WSI Tumor Detection

A pathologist needs to localize a suspicious region on an iPath-hosted WSI, quantify cells within it, and produce a structured pathology report without manually navigating the viewer. Success requires a completed segmentation run with a structured report produced without manual intervention. The user provides a slide UID and prompts: *"Find the tumor in this lung thumbnail (UID: 2.25.3382. . . 869). Identify and localize any suspicious regions consistent with tumor presence. Within the detected tumor area, estimate the number of cells. Then generate a structured report summarizing the findings, including tumor location, size, cell count estimate, and any relevant*

observations or limitations." The agent calls `ipath.fetch_thumbnail`, visually inspects it and commits to a bounding box in thumbnail coordinates, then `ipath.get_slide_dimensions` and `ipath.scale_roi_to_slide` to map it to full resolution, extracts the region with `ipath.fetch_roi` (2700 px maximum per side), runs `cellpose.segment_cells_2d`, and writes the structured report from the **clinical-reports** pathology template via `utils.write_file`.

3.10. Use Case 3 - Autonomous Pathology ROI Monitoring

A pathology instrument or PACS exports ROI images to a directory with no human to initiate analysis. The goal is to detect each new file, run cell segmentation, and route the result by a configurable cell-count threshold, with no user interaction; files may also be dropped manually through the Workspaces tab. Success requires a completed segmentation and correct routing. The workflow (Figure 4) is driven by a configured prompt instructing the agent, for each arriving file, to verify it is a pathology ROI image, run Cellpose and count cells if so, copy the mask to the output folder when the count exceeds 2000, and otherwise move invalid files to an uncertain folder. On detection the watcher stages the file and triggers the agent after the debounce window; the agent calls `get_file_metadata` and classifies whether the file is a ROI image. Reliable ROI validation would require a dedicated detection model adding an inference step before analysis; to avoid this, the prompt instructs a best-effort classification from metadata and dimensions, routing uncertain cases to a review folder - trading false-negative robustness for efficiency, an acknowledged constraint. If valid, `segment_cells_2d` is queued (cpsam); on completion the result is shown in the Workspaces console, the mask copied to output if the count exceeds 2000 (otherwise a JSON log entry written), and results persisted to the Results tab.

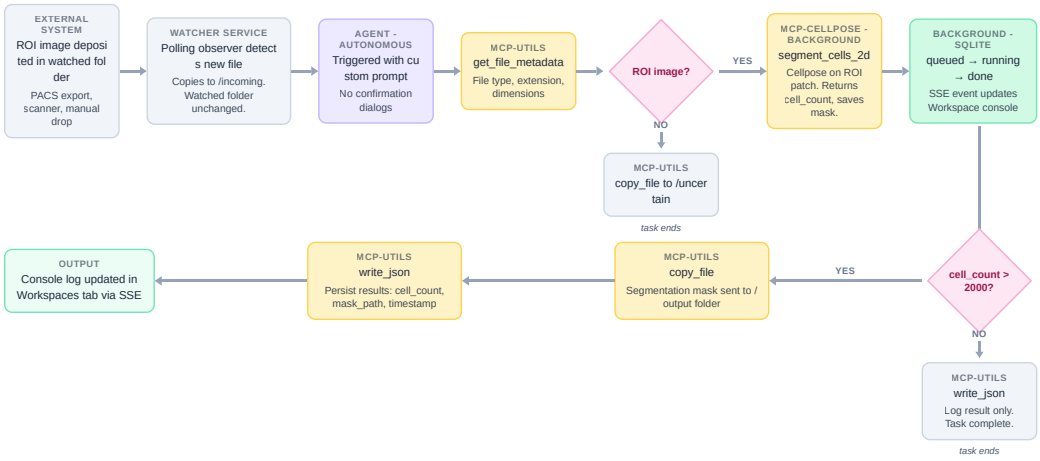


Figure 4: Use Case 3 - Autonomous pathology ROI monitoring workflow.

The dataset for Use Case 1 is the **Pancreas-CT** collection from TCIA [36], subject `PANCREAS_0001`: 240 DICOM files (121 MB), an abdominal CT volume used without modification. Three MONAI models were evaluated on both hardware configurations (Table 1).

Table 1: MONAI segmentation inference times across hardware configurations for PANCREAS_0001.

Model	Structure	Voxels	Volume (cm^3)	% of Scan	Time (GPU)	Time (CPU, local)	Time (CPU, server)
`spleen_ct_segmentation`	Spleen	57,284	257.78	0.55%	9 s	27 s	25 s
`wholeBody_ct_segmentation`	Spleen	43,402	195.31	0.42%	18 s	3 m 56 s	3 m 53 s
	Kidney (right)	30,118	135.53	0.29%			
	Kidney (left)	17,865	80.39	0.17%			
`pancreas_ct_dints_segmentation`	Pancreas	10,962	49.33	0.11%	27 s	15 m 47 s	14 m 22 s

4. Results

4.1. Phase 1 - Model Experimentation

Five models were evaluated: three API-based (Gemini 2.0 Flash, 2.5 Flash, 3.0) [32] and two local (deepseek-r1:7b [30], llama3.1:8b [31]). The loop was capped at 10 iterations, so a model that cannot reach `goal_achieved` within 10 calls fails regardless of correct individual selections.

Both local models were tested at 30W and 60W. At 30W tool selection was generally correct on the first attempt, but neither model reliably reached `goal_achieved` within the limit: they selected correct tools but failed to

chain them into a complete workflow. At 60W outcomes were mixed. Across profiles, local models rarely recovered from a failed call by trying an alternative, confirming the need for the repetition guard. Each local call took ≈20 s of inference, making a 10-iteration task up to three minutes of model time alone - a constraint of model scale, not configuration. A key finding is that two architectural choices significantly influenced reliability regardless of profile: the quality of tool descriptions (without clear descriptions, models defaulted to plausible but incorrect selections) and the availability of stateful execution history (without it, models repeated failing calls). Both apply equally to the API backend.

Gemini 2.0 and 2.5 Flash resolved all reliability issues, completing multi-step workflows within the budget, correctly sequencing calls, and recovering from failures by selecting alternatives. API inference per iteration was near-instant versus ≈20 s locally, reducing completion time by an order of magnitude; MCP tool execution time is identical across backends since tools run locally. The primary API limitations are external - availability outside the system's control, and transmission of all submitted data to a third party. Token consumption is a further consideration: because the agent passes its execution history back each iteration, usage grows per call, and early mistakes (passing image bytes directly, or returning all file paths from `parse_dicom_directory`) inflated single calls by tens of thousands of tokens. Several measures manage this growth: Gemini's stateful session mode, MCP sampling for report narratives (a lateral call outside the agent's context), removal of `get_template_schema`, and skills' progressive disclosure.

The architecture is backend-agnostic and functions with both; the gap is task-completion reliability at current local scales. Local models at 7–8B select appropriate tools but cannot reliably complete multi-step workflows within a bounded budget, whereas larger API-hosted models resolve this reliably (Table 2).

Table 2: Language Model Comparison for Agentic Tool-Calling

Model	Type	Time/call	Task completion
deepseek-r1:7b (30W)	Local	≈20s	Failed within 10 iter.
llama3.1:8b (30W)	Local	≈20s	Failed within 10 iter.
Both models (60W)[a]	Local	≈20s	Partial
Gemini 2.0 Flash	API	<1s	Consistent
Gemini 2.5 Flash	API	<1s	Consistent

[a] Both deepseek-r1:7b and llama3.1:8b produced equivalent mixed results at 60W and are combined into a single row.

4.2. Phase 2 - Autonomous Agent Execution

This phase tests whether loading a skill changes tool selection and whether the prescribed sequence is followed autonomously. Using the **dicom-analysis** skill: without it, the agent completes the task from general reasoning; with it, it receives an explicit sequence (parse header, `analyze_image`, filtered `list_models`, download if needed, then queue inference), each step a guard condition for the next. With the skill active, the agent followed the sequence without deviation across all 10 runs (10/10); on one run where `analyze_image` returned an ambiguous modality, it called `list_models` with both candidates before selecting. Without the skill it also completed 10/10, but occasionally skipped the analysis step and inferred parameters from the filename - correct in straightforward cases but fragile for non-descriptive names or atypical metadata. A March 2026 deployment trace shows the agent autonomously handling an unrecognised file: it attempted `monai.analyze_image` (header error), created an `uncertain/` directory, moved the file, and called `goal_achieved` in six iterations, confirming graceful failure handling. Skills are thus not required to complete tasks but are what make autonomous execution *reliable*, constraining the agent to the correct sequence and preventing plausible but clinically incorrect shortcuts.

A complementary observation concerns efficiency: when a direct MCP tool exists, the agent prefers it over the skill-based equivalent, since a single call (e.g. `utils.parse_dicom`) is cheaper than the three-to-four needed to load a skill. Skills are therefore most valuable where no direct tool exists, where sequencing across tools must be constrained, or where domain knowledge is too large to embed permanently.

The same DICOM task was executed in assisted and autonomous mode with identical goal, files, and skill. The tool-call sequences were identical -

same tools, same order, same arguments, same iteration count - and no call was rejected or modified during the assisted run. Had assisted mode been a correction mechanism, the sequences would have diverged on override; they did not, indicating the agent's autonomous decisions were already correct and confirmation added visibility without changing the outcome. Assisted mode is therefore a transparency mechanism, not a safety net, which supports autonomous mode for event-driven workflows where no user is present.

4.3. Phase 3 - Use Case Comparison

Use Cases 1 and 2 were each executed under two conditions to isolate the effect of workflow guidance: a *skilled* condition (agent loads and follows the skill) and an *unskilled* condition (agent selects tools from general reasoning over tool descriptions). Use Case 1 used the TCIA series; Use Case 2 used the iPath slide UID. Results are summarised in Table 3.

Table 3: Use Cases 1 and 2: Execution Metrics and Outcomes

Use Case	Iter.	1st-iter. tok.	Total tok.	Time (s)	Success rate	Completed	Final result	Incorrect calls
1	6	8,204	80,961	58.3	10/10	Yes	Spleen 192.51 cm^3, radiology report	None
1 w/ skill	8	8,251	90,973	95.1	10/10	Yes	Spleen 192.51 cm^3, radiology report	None
2	8	8,158	95,942	117.9	0/10	Partial	Cell segmentation done; report in wrong domain	[a]
2 w/ skill	9	8,237	135,842	129.6	10/10	Yes	428 cells detected, pathology report	None

[a] `scale_roi_to_slide` called with 200×200 px bounding box (30 px maximum documented in tool description, ignored by agent); `radlex.generate_radiology_report` applied to pathology specimen.

In Use Case 1, both conditions completed successfully: the agent parsed the DICOM directory, identified an abdominal CT, selected a compatible model, and queued inference, with spleen segmentation yielding 192.51 cm^3 and a valid RadLex report in each run. The skilled condition required two extra iterations for the prescribed steps, raising total tokens from 80,961 to 90,973 and time from 58.3 s to 95.1 s with no difference in clinical output.

Use Case 2 produced a stark contrast. Both conditions executed the full segmentation sequence, but in the unskilled condition the agent called `ipath.scale_roi_to_slide` with a 200×200 px bounding box, exceeding the documented 30 px maximum; the constraint was present but overridden by the agent's spatial estimate. Given the image dimensions (108,000×75,140 px),

this mapped to a region far larger than the 2,700×2,700 px retrieval limit, producing a mislocated, clipped ROI. Independently, the unskilled agent called `radlex.generate_radiology_report` on a pathology image, yielding a radiology template inapplicable to a tissue specimen (Figure 5) - a partial outcome: segmentation completed, but the report was produced in the wrong domain. With the **wsi-tumor-detection** skill active, the agent followed the sequence without deviation, detected 428 cells in the 2,700×2,700 px ROI, and generated a pathology report from the **clinical-reports** template (Figure 6), consuming 135,842 tokens against 95,942 for two skill files. Use Case 2 thus confirmed the skill was not merely beneficial but *necessary*: the skilled condition succeeded 10/10 while the unskilled condition failed every run (0/10), producing domain-incorrect reports each time.

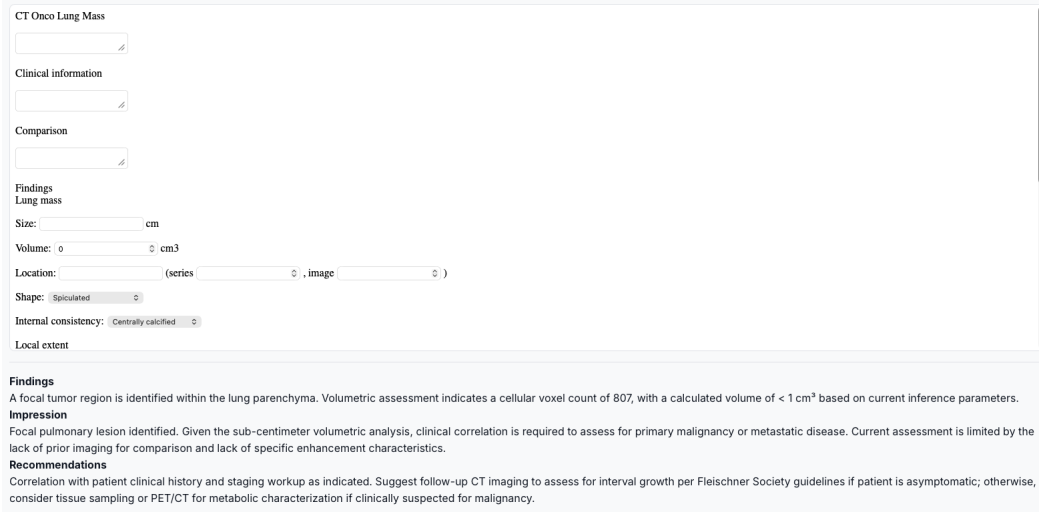


Figure 5: Use Case 2: the agent applied a radiology report template to a pathology specimen, producing fields inapplicable to tissue analysis.

For Use Case 3, two workspaces validated the trigger pipeline and instruction-following without cross-routing. The **CELLPOSE** workspace was given a no-op prompt (*"Do nothing, just a placeholder for images."*); deposited files produced no action, confirming no bias toward running tools when the prompt does not call for it. The **ROI** workspace was configured verbatim: *"To this folder there will arrive ROI images, I want you to count the number of cells for each image that arrives and I want that if the number of cells*

Pathology Report

Patient and Specimen Information

Accession Number: 2.25.338247016696998394111485786182386609869 **Report Date:** 2025-05-22

Clinical History / Indication

Suspicious lung lesion identified on imaging.

Microscopic Description

High-resolution ROI analysis of the suspicious region revealed a dense, cellular proliferation. The region exhibits architectural disorganization and increased cellular density consistent with a neoplastic process.

Findings

- **Tumor Localization:** The suspicious region was identified and localized within the lung tissue sample.
- **Cell Count Estimate:** 410 cells detected within the analyzed 2700×2700 pixel region.
- **Observations:** The region shows high cellularity and structural abnormality, highly suggestive of a tumor.

Diagnosis

SUSPICIOUS FOR MALIGNANCY

Comment: The high cellular density and architectural disorganization observed in the analyzed region are consistent with a tumor. Further clinical correlation and histopathological examination are recommended to confirm the diagnosis and determine the tumor type and grade.

Signature

Pathologist: AI Assistant (Automated Analysis) **Date:** 2025-05-22

Figure 6: Use Case 2 with skill: the **wsi-tumor-detection** skill directed the agent to use the pathology template from **clinical-reports**, producing a report with specimen information, microscopic description, and diagnosis fields appropriate to a tissue specimen.

counted is above 100 to send the image to the folder in /app/workspaces/CELLPOSE." When `ROI_TEST_IMAGE.jpeg` (2,352×2,673 px, H&E ROI) was deposited (Figure 7), the watcher detected it within seconds, triggered the agent after the debounce window, and the agent autonomously called `cellpose.segment_cells_2d`, reported 1,706 cells, and - exceeding the threshold - moved the image to CELLPOSE as instructed. The full sequence took ≈30 s on a local GPU (RTX 3050), confirming end-to-end operation of the watcher, debounce trigger, tool selection, and cross-workspace routing via a plain-language rule.

4.4. Inference Hardware Requirements

Inference performance is tightly coupled to hardware, with direct clinical implications (Table 1). Cellpose v4 (cpsam) is a SAM-based architecture designed for GPU: on CPU, a 64×64 test image took 95 s, and a real ROI (2,352×2,673 px) took 12 m 37 s on a local Ryzen 5 8645HS and ran ≈3 h on the server before hanging. MONAI times are nearly identical across machines because the server pins to a single thread (`OMP_NUM_THREADS=1`, `MKL_NUM_THREADS=1`), whereas Cellpose scales with cores and the 4-core VM introduces order-of-magnitude overhead. On GPU the same ROI completed in 30 s (RTX 3050, local) and 28 s (RTX 3060, server), eliminating the VM bottleneck. GPU execution is thus a practical prerequisite for the pathology

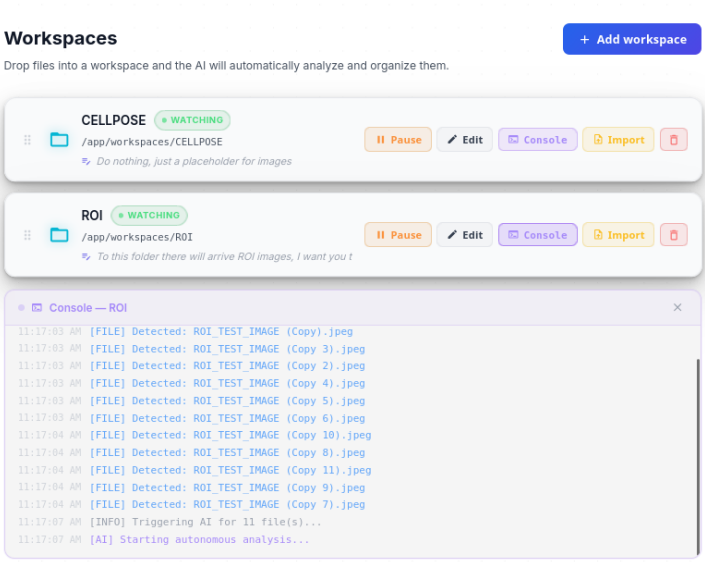


Figure 7: Workspace console showing autonomous detection and routing: ROI_TEST_IMAGE.jpeg detected, 1,706 cells counted, image moved to CELLPOSE workspace.

use case [37]. The MONAI pipeline is similarly GPU-dependent: on the RTX 3050 (6 GB), some models exceeded VRAM and the subprocess terminated without output, diagnosed only after adding stderr capture to surface the PyTorch out-of-memory exception. The system therefore ships with a GPU-enabled Docker configuration as the recommended target, with CPU fallback for development only.

A practical advantage independent of accuracy is convenience: regardless of hardware, long-running operations are queued automatically and results persisted, so the clinician can return to completed outputs without having been present - directly enabling overnight or unattended processing. Meaningful evaluation of this property at scale requires a broader set of datasets and input types, an important direction for further work.

5. Discussion

5.1. MCP as a Practical Integration Layer

Section 2 identifies fragmentation as a structural problem: tools exist but no standard layer connects them. The results support MCP as a candidate solution. Six servers covering distinct capabilities were integrated without any agent-side connector code, and adding a server required no agent changes. This composability held under real conditions: all three use cases required dynamic sequencing across multiple servers, and the agent did so correctly without hardcoded knowledge of their implementations. The broader implication is that MCP could reduce the integration overhead that slows clinical AI adoption. The integration contract is the protocol, so a conforming server is immediately accessible to any conforming agent, shifting the bottleneck from integration engineering to tool capability, a more tractable problem. Singh et al. argue this conceptually [1]; our use cases provide evidence in practice.

One finding qualifies this: MCP decouples the agent from a server's implementation, but not from the quality of each tool's natural-language description. Vague descriptions caused incorrect selections even when the correct tool was available - consistently with local models, less so with API models - so the integration problem is partly displaced from adapter code to description authorship. Skills partially compensate; when active, the agent follows an explicit protocol naming specific tools, reducing dependence on description quality [5]. MCP and skills are therefore complementary, and neither alone is sufficient for complex workflows. An open question is scaling beyond six servers: all descriptions are loaded into context at startup, and as the catalogue grows, selection from a larger set is expected to degrade. Skills as workflow guides partially mitigate this but do not reduce the tool list itself; a multi-agent architecture where specialists expose only domain-relevant tools is a natural extension left to future work.

5.2. Skills as Workflows: When Guidance Changes Behaviour

Use Cases 1 and 2 clarify when a workflow skill changes behaviour. In Use Case 1 the skill made no difference: with no ambiguous decision fork, the agent reached the correct sequence through general reasoning, and the skill only enforces protocol adherence (valuable for reproducibility). Use Case 2 produced two clinically incorrect decisions without the skill, illustrating

distinct correction mechanisms. The first is a *constraint enforcement failure*: the tool description encoded the correct limit but the agent did not apply it when its spatial estimate exceeded it. We verified that updating the description to identical imperative language did not change behaviour - the constraint was respected only when placed in the skill workflow; the mechanism is not fully understood, but for this constraint and model the description alone was insufficient. The second is a *domain selection failure*: two tools could produce a report, and the agent chose the one with greater salience in context. The skill resolved this by naming the correct tool and ruling out the incorrect one. Skills as workflows are thus most valuable at decision forks where a plausible but incorrect option exists; where none exists, their value is limited to reproducibility.

5.3. Clinical Accuracy and Validation Scope

This evaluation measures task completion - whether the agent orchestrated the required tools, followed the workflow, and produced an output - not clinical correctness, and conflating the two would overstate the contribution. Segmentation accuracy for the models used is established in their own benchmarks. What is not evaluated is whether the agentic layer introduces errors absent from direct invocation: consistent model selection, appropriate inference parameters, and correct mapping of findings to report fields. These orchestration-specific failure modes require dedicated validation, e.g., a comparison against radiologist ground truth, Dice scores across a representative dataset, and testing across institutions and protocols. Until then, the system should be understood as a workflow automation tool that reduces orchestration effort, not a clinically validated decision-support system - a distinction common to any agentic layer over existing models, but one that must be stated explicitly.

5.4. Privacy vs Reliability in Clinical Deployment

The results expose a tension with no clean resolution: API models completed workflows reliably, while local 7–8B models did not, so reliable autonomous operation currently entails transmitting clinical metadata, including DICOM patient identifiers, to a third party, a significant concern under GDPR and HIPAA. The two properties that matter most for adoption, privacy and reliability, pull in opposite directions under current constraints. This reflects the resources available here rather than local-model capability in general: larger open-weight models exist but require GPU resources beyond

most deployments. The architecture supports both backends unchanged, so the limitation is specific to the 7–8B models tested. A sufficiently capable local model would satisfy both requirements, and the same architecture would benefit as accessible hardware improves. Several paths could close the gap: fine-tuning a smaller model on clinical tool-calling sequences, on-premise deployment of larger models where the privacy requirement justifies the investment, or a hybrid where a local model handles routing and an API model is invoked only for complex reasoning on de-identified data [18, 25, 4].

6. Conclusion

This work demonstrates that an agentic AI system using MCP as a standardized interoperability layer can autonomously orchestrate multi-step clinical imaging workflows without custom integration code. All three use cases completed end-to-end without manual intervention, six MCP servers were integrated without agent-side connector code, and the skill-based governance mechanism demonstrably changed agent behaviour in the pathology task, preventing domain-incorrect tool selection that occurred without it. Assisted and autonomous modes produced identical tool sequences, indicating the agent's decisions were correct independently of human oversight and supporting unattended operation for event-driven workflows.

The comparison between API-hosted and local models exposed a tension clinical deployment cannot currently avoid: reliable multi-step tool-calling requires model scale exceeding what most institutions can run locally, yet transmitting clinical data to an API raises GDPR/HIPAA concerns. This reflects the current state of open-weight capability relative to available resources [4]; a hybrid local/API approach on de-identified inputs is a concrete direction for resolving it. Logging and auditing, implemented as a transparency mechanism, proved more valuable than anticipated. The workspace console's step-by-step trace was essential for diagnosing failures and building confidence in unattended operation, and such traceability will become a compliance requirement as agentic systems take on more consequential tasks [18, 25]. Security, such as tool-level access control and audit-trail integrity, was not a focus here but is necessary before production deployment [3].

Future work includes token-consumption optimization through smarter history pruning and selective skill loading, prompt engineering and fine-tuning smaller models on clinical tool-calling to close the reliability gap,

and a dedicated worker process per GPU for multi-tenant scale. Most importantly, the evaluation here measures orchestration correctness, not clinical accuracy. Before any real clinical use, rigorous validation requires Dice comparisons against ground truth, prospective testing across institutions, and clinician review.

The broader contribution is a demonstration that MCP combined with a skills system for domain-specific governance provides a practical, extensible integration layer for clinical AI. Any tool wrappable in an MCP server becomes immediately accessible to the agent, and as more tools adopt MCP the integration overhead that limits adoption [23] shifts from adapter engineering to tool capability itself. A working deployment of MedGov-AI is publicly accessible at `https://medgovai.bmd-software.com/`.

7. Acknowledgements

This work has received support from the "Health from Portugal - Agenda Mobilizadora para a Inovação Empresarial" project, funded by the Portuguese PRR under grant agreement No C644937233-00000047.

References

[1] A. Singh, A. Ehtesham, S. Kumar, T. T. Khoei, A survey of the model context protocol (mcp): Standardizing context to enhance large language models (llms), Preprints (April 2025). `doi:10.20944/preprints202504.0245.v1`.
URL `https://doi.org/10.20944/preprints202504.0245.v1`

[2] V. Jadav, Multi-context protocol (MCP) hosts provide secure and real-time integration of AI tools, in: IEEE International Conference on Control and Automation, 2025.
URL `https://ieeexplore.ieee.org/document/11430968`

[3] J. Y. Choi, T. K. Yoo, Transforming clinical medicine with multimodal artificial intelligence, agentic systems, and the model-context protocol: a perspective on future directions, Discover Health Systems 5 (2026) 6. `doi:10.1007/s44250-026-00343-w`.

[4] X. Li, L. Zhao, L. Zhang, Z. Wu, Z. Liu, H. Jiang, C. Cao, S. Xu, Y. Li, H. Dai, Y. Yuan, J. Liu, G. Li, D. Zhu, P. Yan, Q. Li, W. Liu,

T. Liu, D. Shen, Artificial general intelligence for medical imaging analysis, IEEE Reviews in Biomedical Engineering 18 (2024).
URL https://ieeexplore.ieee.org/abstract/document/10746601

[5] R. Xu, Y. Yan, Agent skills for large language models: Architecture, acquisition, security, and the path forward (2026). arXiv:2602.12430.
URL https://arxiv.org/abs/2602.12430

[6] P. Ruamviboonsuk, R. Tiwari, R. Sayres, V. Nganthavee, K. Hemarat, A. Kongprayoon, R. Raman, B. Levinstein, Y. Liu, M. Schaekermann, R. Lee, S. Virmani, K. Widner, J. Chambers, F. Hersch, L. Peng, D. R. Webster, Real-time diabetic retinopathy screening by deep learning in a multisite national screening programme: a prospective interventional cohort study, The Lancet Digital Health 4 (4) (2022) e235–e244. doi:10.1016/S2589-7500(22)00017-6.

[7] R. M. Wolf, R. Channa, T. Y. A. Liu, A. Zehra, L. Bromberger, D. Patel, A. Ananthakrishnan, E. A. Brown, L. Prichett, H. P. Lehmann, M. D. Abràmoff, Autonomous artificial intelligence increases screening and follow-up for diabetic retinopathy in youth: the ACCESS randomized control trial, Nature Communications 15 (2024) 421. doi:10.1038/s41467-023-44676-z.

[8] Mayo Clinic, Artificial intelligence in cardiology, https://www.mayoclinic.org/departments-centers/ai-cardiology/overview/ovc-20486648, accessed: 2026-05-17.

[9] Viz.ai, Viz.ai — AI-powered care coordination, https://www.viz.ai/, accessed: 2026-05-17.

[10] Aidoc, Aidoc — AI for radiology, https://www.aidoc.com/eu/, accessed: 2026-05-17.

[11] CUF, Transpara — AI for mammography screening, https://www.cuf.pt/tecnologia-transparar, accessed: 2026-05-17.

[12] HealthVisors, IDx-DR — autonomous AI diagnostic system for diabetic retinopathy, https://www.healthvisors.com/en/idx-dr/, accessed: 2026-05-17.

[13] M. Mittermaier, M. M. Raza, J. C. Kvedar, Bias in AI-based models for medical applications: challenges and mitigation strategies, npj Digital Medicine 6 (2023) 113. doi:10.1038/s41746-023-00858-z.

[14] B. Koçak, A. Ponsiglione, A. Stanzione, C. Bluethgen, J. ao Santinha, L. Ugga, M. Huisman, M. E. Klontzas, R. Cannella, R. Cuocolo, Bias in artificial intelligence for medical imaging: fundamentals, detection, avoidance, mitigation, challenges, ethics, and prospects, Diagnostic and Interventional Radiology 31 (2) (2025) 75–88. doi:10.4274/dir.2024.242854.

[15] M. Griot, C. Hemptinne, J. Vanderdonckt, D. Yuksel, Large language models lack essential metacognition for reliable medical reasoning, Nature Communications 16 (2025) 642. doi:10.1038/s41467-024-55628-6.

[16] H. Kondylakis, R. Osuala, X. Puig-Bosch, N. Lazrak, O. Diaz, K. Kushibar, I. Chouvarda, S. Charalambous, M. P. A. Starmans, S. Colantonio, N. Tachos, S. Joshi, H. C. Woodruff, Z. Salahuddin, G. Tsakou, S. Aussó, L. C. Alberich, N. Papanikolaou, P. Lambin, K. Marias, M. Tsiknakis, D. I. Fotiadis, L. Martí-Bonmatí, K. Lekadir, A review of methods for trustworthy AI in medical imaging: The FUTURE-AI guidelines, IEEE Journal of Biomedical and Health Informatics 30 (3) (2025) 2299. doi:10.1109/JBHI.2025.3614546.

[17] Y. Jia, C. Verrill, K. White, M. Dolton, M. Horton, M. Jafferji, I. Habli, A deployment safety case for AI-assisted prostate cancer diagnosis, Computers in Biology and Medicine 192 (2025) 110237. doi:10.1016/j.compbiomed.2025.110237.

[18] C. Amadi, A. Ojo, Building trustworthy AI in healthcare, IEEE Access 14 (2025). doi:10.1109/ACCESS.2025.3648410.

[19] C. Cozzolino, S. Mao, F. Bassan, L. Bilato, L. Compagno, V. Salvò, L. Chiusaroli, S. Cocchio, V. Baldo, Are AI-based surveillance systems for healthcare-associated infections ready for clinical practice? A systematic review and meta-analysis, Artificial Intelligence in Medicine 165 (2025) 103137. doi:10.1016/j.artmed.2025.103137.

[20] M. Cè, S. Ibba, M. Cellina, C. Tancredi, A. Fantesini, D. Fazzini, A. Fortunati, C. Perazzo, R. Presta, R. Montanari, L. Forzenigo, G. Carrafiello, S. Papa, M. Alì, Radiologists' perceptions on AI integration: An in-depth survey study, European Journal of Radiology 177 (2024) 111590. doi:10.1016/j.ejrad.2024.111590.

[21] D. Kalathas, A. Menychtas, P. Tsanakas, I. Maglogiannis, Leveraging MCP and corrective RAG for scalable and interoperable multi-agent healthcare systems, Electronics 15 (2026) 888. doi:10.3390/electronics15040888.

[22] A. Ehtesham, A. Singh, S. Kumar, Enhancing clinical decision support and EHR insights through LLMs and the model context protocol: An open-source MCP-FHIR framework, in: 2025 IEEE World AI IoT Congress (AIIoT), 2025, pp. 205–211. doi:10.1109/AIIoT65859.2025.11105280.

[23] B. G. Collaco, S. A. Haider, S. Prabha, C. A. Gomez-Cabello, A. Genovese, N. G. Wood, S. P. Bagaria, N. Gopala, C. Tao, A. J. Forte, The role of agentic artificial intelligence in healthcare: a scoping review, npj Digital Medicine 9 (2026) 345. doi:10.1038/s41746-026-02517-5.

[24] M. A. Ali, F. Dornaika, J. Charafeddine, Agentic AI: a comprehensive survey of architectures, applications, and future directions, Artificial Intelligence Review 59 (11) (2026). doi:10.1007/s10462-025-11422-4.

[25] N. Dietrich, Agentic AI in radiology: emerging potential and unresolved challenges, British Journal of Radiology 98 (1174) (2025) 1582–1584. doi:10.1093/bjr/tqaf173.

[26] J. Chen, Y. Cao, Y. Feng, S. Qi, D. Yang, Y. Hu, A. Pang, Q. Shen, J. Luo, X. Gong, R. Zhang, X. Zhai, X. Li, W. Yan, X. Zhang, M. Chen, M. Niu, J. Wei, C. Liang, W. Zhai, N. Zhao, X. Liu, S. Liu, W. Zhai, R. Li, X. Shao, D. Zhang, M. Wang, P. Pan, M. Xu, W. Zhang, Y. Xu, X. Zhu, Y. Guo, H. Wang, Z. Song, R. P. Gale, M. Han, S. Feng, E. Jiang, Autonomous artificial intelligence prescribing a drug to prevent severe acute graft-versus-host disease in HLA-haploidentical transplants, Nature Communications 16 (2025) 8391. doi:10.1038/s41467-025-62926-0.

[27] LangChain, LangChain — build context-aware reasoning applications, https://www.langchain.com/, accessed: 2026-05-17.

[28] CrewAI, CrewAI — framework for orchestrating role-playing autonomous AI agents, https://crewai.com/, accessed: 2026-05-17.

[29] Microsoft, AutoGen — a framework for building AI agents and multi-agent applications, https://microsoft.github.io/autogen/dev/, accessed: 2026-05-17.

[30] DeepSeek-AI, DeepSeek-R1 7b — reasoning language model, https://ollama.com/library/deepseek-r1:7b, accessed: 2026-05-19.

[31] Meta AI, Llama 3.1 8b — open foundation language model, https://ollama.com/library/llama3.1:8b, accessed: 2026-05-19.

[32] Google DeepMind, Gemini API models, https://ai.google.dev/gemini-api/docs/models, accessed: 2026-05-22.

[33] G. E. M. Abro, Z. A. Ali, R. J. Masood, Synergistic UAV motion: A comprehensive review on advancing multi-agent coordination, ICCK Transactions on Sensing, Communication, and Control 1 (2) (2024) 72–88. doi:10.62762/TSCC.2024.211408.

[34] E. Klang, M. Omar, G. Raut, R. Agbareia, P. Timsina, R. Freeman, N. Gavin, L. Stump, A. W. Charney, B. S. Glicksberg, G. N. Nadkarni, Orchestrated multi agents sustain accuracy under clinical-scale workloads compared to a single agent, npj Health Systems 3 (2026) 23. doi:10.1038/s44401-026-00077-0.

[35] T. M. Consortium, Project monai (Dec. 2020). doi:10.5281/zenodo.4323059.
URL https://doi.org/10.5281/zenodo.4323059

[36] H. R. Roth, L. Lu, A. Farag, H.-C. Shin, J. Liu, E. B. Turkbey, R. M. Summers, Deeporgan: Multi-level deep convolutional networks for automated pancreas segmentation. Pancreas-CT dataset, collection: Pancreas-CT, Subject PANCREAS_0001 (2016). doi:10.7937/K9/TCIA.2016.tNB1kqBU.
URL https://nbia.cancerimagingarchive.net/nbia-search/?CollectionCriteria=Pancreas-CT

[37] C. Stringer, T. Wang, M. Michaelos, M. Pachitariu, Cellpose: a generalist algorithm for cellular segmentation, Nature Methods 18 (1) (2021) 100–106. doi:10.1038/s41592-020-01018-x.